

Investigación Comprehensive: Modernización de Plataformas QUBE Servo

Arquitectura Propuesta: ESP32 + LM2596 + INA219 + L298N

Investigación de viabilidad técnica

2026-05-11

Índice

1. Investigación Comprehensive: Modernización de Plataformas QUBE Servo	4
1.1. Arquitectura Propuesta: ESP32 + LM2596 + INA219 + L298N	4
1.2. RESUMEN EJECUTIVO	4
1.2.1. Pregunta Central	4
1.2.2. Respuesta Directa	4
1.3. PARTE 1: ESTADO DEL ARTE EN GITHUB	4
1.3.1. 1.1 Proyectos de Control PID de Motores DC + L298N	4
1.3.2. 1.2 Proyectos de ESP32 + Control de Motor DC	4
1.3.3. 1.3 Proyectos INA219 + Monitoreo de Potencia	5
1.3.4. 1.4 Proyectos de Sistemas Educativos Rotatorios (similares a QUBE)	6
1.3.5. 1.5 Proyectos de Buck Converter (LM2596)	6
1.4. PARTE 2: ANÁLISIS COMPARATIVO DE ARQUITECTURAS	6
1.4.1. 2.1 Arquitectura Propuesta vs Existentes	6
1.4.2. 2.2 Comparación con Alternativas	7
1.5. PARTE 2.5: DIAGNÓSTICO E IMPLEMENTACIÓN REAL — HW-FIX-1 (Encoder Signal Conditioning)	7
1.5.1. 2.5.1 Problema de Campo Identificado	7
1.6. PARTE 3: DESAFÍOS TÉCNICOS REPORTADOS	9
1.6.1. 3.1 Desafíos en Proyectos Similares	9
1.6.2. 3.2 Problemas Potenciales en Arquitectura Propuesta	10
1.7. PARTE 4: REFERENCIAS ACADÉMICAS	11
1.7.1. Papers Académicos Relevantes (mencionados en GitHub)	11
1.7.2. Libros Recomendados (Implícitos en el código)	11
1.8. PARTE 5: COMBINACIONES SIMILARES ENCONTRADAS	11
1.8.1. 5.1 “Closest Match” Arquitecturas	11
1.9. PARTE 6: VIABILIDAD TÉCNICA DETALLADA	12
1.9.1. 6.1 Selección de Componentes	12
1.9.2. 6.2 Matriz de Pines ESP32	13
1.9.3. 6.2.1 Conexion Pin por Pin (final — post puesta en marcha 2026-04-29)	14
1.9.4. 6.2.2 Acondicionamiento de señal del encoder (open-drain — confirmado 2026-05-07)	14
1.9.5. 6.3 Cronograma de Implementación	15
1.10. PARTE 7: CONCLUSIONES Y RECOMENDACIONES	16
1.10.1. 7.1 ¿Alguien ha hecho EXACTAMENTE esto?	16
1.10.2. 7.2 ¿Qué combinaciones similares SÍ existen?	16
1.10.3. 7.3 Desafíos Técnicos Reportados por la Comunidad	17
1.10.4. 7.4 Referencias Académicas que Sustentan la Solución	17
1.10.5. 7.5 Validación de Arquitectura Propuesta	17

1.11. PARTE 8: RECOMENDACIONES PRÁCTICAS	18
1.11.1. 8.1 Para Iniciar el Proyecto	18
1.11.2. 8.2 Recursos Recomendados	18
1.12. PARTE 9: RIESGOS Y LIMITACIONES	19
1.12.1. 9.1 Riesgos Identificados	19
1.12.2. 9.2 Limitaciones Conocidas	19
1.13. CONCLUSIÓN FINAL	19
1.13.1. ¿Es viable la arquitectura ESP32 + LM2596 + INA219 + L298N?	19
1.13.2. SÍ, es completamente viable y recomendable	19
1.14. REFERENCIAS BIBLIOGRÁFICAS RECOPIADAS	19
1.14.1. Papers y documentos académicos	20
1.14.2. Repositorios GitHub	20
1.14.3. Librerías Arduino / ESP32	20
1.14.4. Datasheets	20
1.15. PARTE 10: Integración Detallada para Tesis	21
1.15.1. 9.1 Contexto y Motivación Académica	21
1.15.2. 9.2 Objetivos Integrados	21
1.15.3. 9.3 Metodología Experimental Unificada	21
1.15.4. 9.4 Métricas de Evaluación Recomendadas	22
1.15.5. 9.5 Riesgos y Mitigaciones	22
1.15.6. 9.6 Criterios de Éxito	23
1.15.7. 9.7 Hoja de Ruta de Cierre Académico	23
1.16. APÉNDICE A: Comandos de Utilidad	23
1.16.1. Búsquedas en GitHub Realizadas	23
1.16.2. Configuración I2C para INA219 en ESP32	23
1.16.3. Configuración PWM + Encoder en ESP32	24
1.17. APÉNDICE B: Diagramas de Hardware — Implementación Final (2026-05-06/07)	24
1.17.1. B.1 Diagrama de Conexión INA219 → ESP32	24
1.17.2. B.2 Filtros RC para Señales de Encoder (sin Schmitt trigger)	25
1.17.3. B.3 Diagrama de Alimentación — LM2596 → ESP32 + L298N	26
1.17.4. B.4 Conexión de Alimentación ESP32 desde LM2596	26
1.17.5. B.5 Diagrama de Sistema Completo (post puesta en marcha 2026-05-06)	27
1.18. APÉNDICE C: Diagramas Eléctricos Consolidados (Final)	27
1.18.1. C.1 Diagrama eléctrico de potencia (fuente, sensado y etapa de motor)	27
1.18.2. C.2 Diagrama eléctrico de control y lógica (ESP32, L298N, INA219)	27
1.18.3. C.3 Diagrama eléctrico del encoder open-drain (implementación validada)	28
1.18.4. C.4 Tabla consolidada de conexiones pin a pin	28

Índice de figuras

Índice de cuadros

hispagestyle{empty}

Investigación Comprehensive: Modernización de Plataformas QUBE Servo

Arquitectura Propuesta: ESP32 + LM2596 + INA219 + L298N

Documento técnico para integración a tesis

Fecha de investigación base: Abril 27, 2026

Última actualización: Mayo 11, 2026

Autor: Investigación de viabilidad técnica

1. Investigación Comprehensive: Modernización de Plataformas QUBE Servo

1.1. Arquitectura Propuesta: ESP32 + LM2596 + INA219 + L298N

Fecha de Investigación: Abril 27, 2026

Autor: Investigación de viabilidad técnica

Objetivo: Validación de arquitectura de modernización de QUBE Servo usando microcontroladores modernos

1.2. RESUMEN EJECUTIVO

1.2.1. Pregunta Central

¿Existe algún proyecto que haya implementado exactamente: **ESP32 + LM2596 (buck converter) + INA219 (telemetría) + L298N (motor driver)** para control rotatorio educativo?

1.2.2. Respuesta Directa

NO. No se encontraron proyectos que combinen exactamente estos cuatro componentes en esa arquitectura específica. Sin embargo: - Existen 41+ proyectos de ESP32 + control DC motor + PID - Existen 74+ proyectos de INA219 + monitoreo de potencia - Existen 44+ proyectos de buck converters (LM2596 es estándar) - Existen 20+ sistemas educativos de péndulo rotatorio (similares a QUBE) - La combinación ES VIABLE pero INÉDITA

1.3. PARTE 1: ESTADO DEL ARTE EN GITHUB

1.3.1. 1.1 Proyectos de Control PID de Motores DC + L298N

Repositorio	Autor	Actualización	Descripción	Similitud
ROS_Arduino_PID_Controlling_DC_Motors	DekeCosta	Jun 2020	Arduino + ROS + L298N + 2 motores DC	Media (ROS overkill)
self-balancing-bot	osasinrobotics	18 días atrás	Arduino + PID + L298N + MPU6050	Media (sin encoder)
arduino_pid_controlled_motor	rolly	Feb 2025	Arduino + L298N + Encoder + PID cerrado	ALTA (pero Arduino, no ESP32)
Speed-Control-of-DC-Motor-Using-Arduino-and-L298N	Hagar633	Enero 2025	Arduino + L298N + Encoder incremental	ALTA (validación educativa)

Hallazgo: Existen casos bien documentados de Arduino + L298N + encoder feedback, pero con **microcontroladores antiguos (Arduino Uno)**.

1.3.2. 1.2 Proyectos de ESP32 + Control de Motor DC

Repositorio	Autor	Stars	Año	Descripción	Evaluación
Esp32CameraRover2	Ver2	46	2018-2024	Framework diferencial drive, closed-loop speed control, encoder feedback, web interface	EXCELENTE
ESP32_Motor_control	amir	0	2025	Dos motores + encoders ópticos + RTOS + parámetros persistentes	BUENA
PID-Motor-Controller	beanjamminb	0	2025 (reciente)	ESP32 + TB6612FNG + PID + encoder planning	BUENA
Speed-Control-Of-DC-Motor	Chaitanya0523	0	2025	PWM básico sin feedback	Básica
Motor_Speed_PID	BlagojeBlagojevic	7	2024	PID implementado (referencia)	Media

Hallazgo Principal: EzRover (46 stars) es el más relevante. Implementa: - Closed-loop speed control con encoders - Odometría y estimación de pose - Go-to-goal behavior - Web interface + WebSocket - Usa L9110S (no L298N, pero similar H-bridge)

1.3.3. 1.3 Proyectos INA219 + Monitoreo de Potencia

Repositorio	Tipo	Stars	Descripción
Adafruit_INA219	Arduino Library	229	Librería oficial Adafruit (C++, I2C)
INA219_WE	Arduino Library	53	Alternativa a Adafruit, bien documentada
RobTillaart/INA219	Arduino Library	32	Librería Arduino estándar (licencia MIT)
pi_ina219	Python (RPi)	117	Soporte Raspberry Pi + I2C (Nov 2023)
ros-power-ina219	ROS Node	10	Integración con ROS
INA (Zanduino)	Arduino Library	168	Librería múltiples INA2xx (más robusta)

Hallazgo: INA219 está MUY bien soportado en Arduino/ESP32. No hay barrera técnica.

1.3.4. 1.4 Proyectos de Sistemas Educativos Rotatorios (similares a QUBE)

Repositorio	Tipo	Hardware	Año	Descripción
Rotary-Inverted-Pendulum	MATLAB+Arduino	Arduino Uno + L298N + encoder	2023	REFERENCIA ACADÉMICA: LQR control, hardware-in-loop, Simulink
RotaryPendulumControl	MATLAB	-	2023	Diseño de controlador con observador de estado
Applied-Control-Systems-Module	LabVIEW	-	2023	Proyecto ELEC6228 (LQR, MPC)
Motor-Pendulum-Control	MATLAB/Simulink	-	2025	Modelado y control PID de péndulo rotatorio
cv-rotary-pendulum-control	Python/OpenCV	-	2024	Visión computacional para control
Rotary-Inverted-Pendulum-	MATLAB	-	2023	Sistema de control con análisis de balance

Hallazgo Crítico: El proyecto **ebrahimabdelghfar/Rotary-Inverted-Pendulum** (2023) es una VALIDACIÓN ACADÉMICA completa que usa Arduino + L298N + encoder, documentando: - Modelado matemático (state space) - Estimación de parámetros del motor - Control LQR implementado - Hardware-in-loop en Simulink - **Limitaciones reportadas:** Ángulo $\pm 20^\circ$, sin swing-up, con vibración

1.3.5. 1.5 Proyectos de Buck Converter (LM2596)

Repositorio	Descripción	Características
OpenCircuitt/Power_Supply_Wifi	LM2596 Buck Converter salida ESP8266 + XL4015 buck converter (2021)	HTML + esquemas (2025) Control WiFi del power supply
pranjals94/BuckConverterPowerSupply	LM2596 + bootstrap driving N-channel MOSFET	Control programado

Conclusión LM2596: Disponible, maduro, bien documentado. No hay barrera técnica.

1.4. PARTE 2: ANÁLISIS COMPARATIVO DE ARQUITECTURAS

1.4.1. 2.1 Arquitectura Propuesta vs Existentes

ARQUITECTURA PROPUESTA

ENTRADA: 12-24V (batería o PSU)

↓

[LM2596 Buck Converter] → 5V/3.3V para ESP32 + sensores

↓

[ESP32-WROOM-32]

ADC (GPIO 34-39): Entrada encoder/sensor

GPIO 26/27: Control de dirección y PWM al L298N

I2C (GPIO 21=SDA, GPIO 22=SCL): INA219

UART (GPIO 1=TX, GPIO 3=RX): Depuración/telemetría

↓

[L298N Motor Driver]

IN1/IN2: Dirección + PWM (GPIO26/27, modo sin ENA cableado)

ENA: Jumper habilitado (sin conexión al ESP32)

OUT1, OUT2: Motor DC

↓

[INA219 Current Sensor] (I2C)

Medición: V, I, P del motor

Dirección I2C: 0x40-0x4F (configurable)

Rango típico: $\pm 3.2A$ @ $\pm 0.320V$

↓

[Encoder + Motor DC]

Pulsos A/B → GPIO ESP32 + interrupción

1.4.2. 2.2 Comparación con Alternativas

Aspecto	ESP32+LM2596+INA219+L298N	Arduino+LM2596+INA219+L298N	Quanser QUBE
Costo	\$40-60 USD	\$30-40 USD	\$2,500-3,500 USD
Conectividad	WiFi + BLE nativa	Opcional (shield)	Ethernet
Procesamiento	Dual-core, 240MHz	Single-core, 16MHz	DSP dedicado
Telemetría de	INA219 (I2C)	Shunt resistor + ADC	Sensores integrados
Potencia			
Closed-loop PID	Software ESP32	Software Arduino	Hardware/Software
Curva de aprendizaje	Media	Baja	Alta (Matlab)
Comunidad/Soporte	Muy grande (Arduino IDE)	Grande	Pequeña (académica)

1.5. PARTE 2.5: DIAGNÓSTICO E IMPLEMENTACIÓN REAL — HW-FIX-1 (Encoder Signal Conditioning)

1.5.1. 2.5.1 Problema de Campo Identificado

Durante las pruebas de hardware (2026-04-27 a 2026-04-29), se observó que **el encoder del servo (Pre-motec 990412016913) no proporcionaba lecturas confiables**, a pesar de que el firmware y motor funcionaban correctamente. Los síntomas fueron:

- Contador de encoder (CNT) permanecía estático o actualizaba esporádicamente aunque el eje girase continuamente
- Posición reportada (POS) no variaba
- Señales A/B en GPIO34/GPIO35 parecían “congeladas” en niveles intermedios ($\sim 1.5V$)
- El lazo PID tenía retroalimentación deficiente, causando que m2 (modo PID) no convergiera

1.5.1.1. Root Cause Analysis Causas Raíz Identificadas:

1. Topología de Open-Drain del Encoder

- El encoder Premotec 990412016913 tiene **salida open-drain** (NPN), no push-pull.
- Estado neutro: línea flota (sin pull-up) 2.5 V (indeterminado)
- Estado activo: transistor interno conduce $\rightarrow$ 0 V (GND)
- Nunca genera “1” activamente; depende de resistencia externa para alcanzar “1”

2. Level Shifter Inadecuado (7 M Ω de impedancia)

- El circuito de acondicionamiento de señal original incluía un **level shifter para convertir 5 V $\rightarrow$ 3.3 V**
- Este componente tenía **impedancia de salida extremadamente alta (~7 M Ω)**
- Con capacitancia parasita del cableado (50–100 pF típico), la **constante de tiempo RC 350–700 μ s** era demasiado lenta para transitorios de encoder (períodos de edge ~5–20 μ s a 2048 CPR $\times$ motor speed)
- Resultado: **los flancos se suavizaban**, quedando el signal en ~1.5 V (entre 0 y 3.3 V)
- El ESP32 (GPIO34/GPIO35) **no podía discriminar** si era “0” o “1” en esa zona de incertidumbre

3. GPIO34/GPIO35 Input Thresholds

- GPIO34 y GPIO35 del ESP32 no tienen pull-ups internos
- Threshold típicamente: ~1.65 V (pero puede variar con temperatura/proceso)
- A 1.5 V, el GPIO queda en zona de incertidumbre $\rightarrow$ lee alternadamente 0 y 1 $\rightarrow$ ruido

1.5.1.2. Solución Implementada Eliminación del Level Shifter + Pull-ups Directos a 3.3 V

ANTES (No Funcionaba):

```
Encoder      (open-drain, salida ~2.5V idle, 0V active)
A / B
```

[Level Shifter 7M Ω] $\leftarrow$ RC charging too slow!

$\rightarrow$ GPIO34/35 @ ESP32 (lee 1.5V $\rightarrow$ indeterminado)

DESPUÉS (Funciona):

```
Encoder      (open-drain, salida 0V/Hi-Z)
A / B
```

4.7k Ω (pull-up a 3.3V)

+3.3V (VDD)

$\rightarrow$ GPIO34/35 @ ESP32 (lee 0V o 3.3V, limpio)

Valores de Componentes Finales: - Pull-up resistor: **4.7 k Ω** (rango típico 4.7–10 k Ω ; 4.7 k Ω elegido para transitorios rápidos) - Voltaje de pull-up: **3.3 V** (no 5 V, compatible con ESP32 logic levels) - Topología: **Pull-up pasivo** (sin level shifter, sin buffers)

1.5.1.3. Validación Comportamiento después del fix: - CNT aumenta monótonamente mientras gira el eje (antes: estático) - POS refleja rotación continua en tiempo real - Decodificación cuadratura X4 funciona (QUAD_LUT produce transiciones correctas) - Telemetría JSON muestra **enc_a** y **enc_b**

alternando entre 0 y 1 limpiamente - Lazo PID ahora recibe retroalimentación confiable → m2 converge a setpoint

Telemetría de Comparación:

Métrica	Antes (Level Shifter)	Después (Pull-ups)
enc_a	Alternando rápido (ruido)	Transiciones limpias cada ~50 ms
enc_b	Idem	Idem
CNT	±0 cambio por minuto	+2048 counts/revolución
POS	0.0° (fijo)	0° → 360° → 0° (continuo)
PID convergence	No (sin feedback)	Sí (típico ±2° en 2-3 seg)

1.5.1.4. Lecciones Técnicas 1. Open-Drain Output Design - Open-drain requiere pull-up de impedancia baja (~1-10 kΩ típico) - No tolera impedancias altas (level shifters mal diseñados, buffers con salida de alta impedancia) - Constante de tiempo RC debe ser **< 10 % del período de transición esperado**

2. Level Shifter Pitfalls - Los level shifters resistivos de alta impedancia (> 1 MΩ) degradan señales digitales de alta velocidad - Para open-drain → CMOS (5V → 3.3V), la topología correcta es: - Opción A: Pull-up directo a 3.3V (si 0V de open-drain es tolerable en destino) - Opción B: Transistor buffer activo (BJT/MOSFET) con pull-up - Opción C (high-impedance resistive shifter): No recomendada para transitorios rápidos

3. GPIO Input Design - GPIO34/35 de ESP32 son **ADC-dedicated, input-only pins** - No tienen pull-ups internos (a diferencia de GPIO con capacidad de salida) - Requieren manejo cuidadoso de impedancia de señal externa - Usarlos para encoders digitales es posible pero requiere condicionamiento previo

4. PCB Layout Implications - Capacitancia parasita es enemiga de slew rate rápido - Minimizar longitud de cable encoder (> 1 m empieza a degradar transitorios) - Retornar shield del encoder al GND de señal en un solo punto (star-grounding)

1.5.1.5. Cambios de Firmware Ninguno requerido. El firmware ya usaba:

```
pinMode(PIN_ENC_A, INPUT); // Correcto: no fuerza pull-up (lo hace la resistencia externa)
pinMode(PIN_ENC_B, INPUT);
```

1.5.1.6. Cambios de Hardware Conexión Revisada - Encoder Servo:

Conector Pin	Señal	Destino	Componente	Resistor
1	GND	ESP32 GND	-	-
2	Encoder A	GPIO34	4.7 kΩ pull-up	Sí (a 3.3V)
3	Encoder B	GPIO35	4.7 kΩ pull-up	Sí (a 3.3V)
4	+5V	-	NO usar	-

Modificaciones realizadas: - Removido level shifter 5V→3.3V (fue la causa del problema) - Instalados resistores 4.7 kΩ en A y B - Conectados a +3.3V (VDD del ESP32)

1.6. PARTE 3: DESAFÍOS TÉCNICOS REPORTADOS

1.6.1. 3.1 Desafíos en Proyectos Similares

1.6.1.1. Del proyecto Rotary-Inverted-Pendulum (ebrahimabdelghfar, 2023):

1. **Vibración del sistema** - Especialmente a alta velocidad

- **Causa:** Inercia del motor + L298N switching
- **Solución:** Filtrado PWM, capacitores (100 μ F) en salida L298N
- 2. **Rango de control limitado** - $\pm 20^\circ$ (fuera de este rango sistema off)
 - **Causa:** Saturación del controlador LQR
 - **Solución:** Anti-windup, limitadores de velocidad
- 3. **Sin funcionalidad swing-up**
 - **Causa:** Control LQR es estabilización, no energía de entrada
 - **Solución:** Agregar controlador mode-switching (swing-up + LQR)
- 4. **Estimación de parámetros del motor**
 - **Proceso:** Excitar motor $\pm 12V$, registrar V y rad/sec
 - **Herramienta:** MATLAB Parameter Estimator (Simulink)
 - **Duración:** 1-2 horas de calibración

1.6.1.2. Del proyecto EzRover (Ezward, 2018-2024):

1. **Odometría con encoders ópticos**
 - **Solución implementada:** Contadores de interrupción en tiempo real
 - **Precisión:** Dependiente de PPR (típicamente 20 PPR)
 2. **Control diferencial de velocidad**
 - **Desafío:** Skew entre ruedas (asimetría mecánica)
 - **Solución:** Calibración de ganancia K_{left} vs K_{right}
 3. **Latencia de comunicación**
 - **Solución:** Tasks independientes RTOS para encoder + control
-

1.6.2. 3.2 Problemas Potenciales en Arquitectura Propuesta

1.6.2.1. Problema 1: Resolución del Encoder

- **Encoder típico:** 20-100 PPR
- **A 3000 RPM:** ~ 1000 pulsos/sec
- **Con interrupción GPIO ESP32:** Fácilmente manejable (máx $\sim 10kHz$)
- **Solución:** Timer/Contador hardware del ESP32

1.6.2.2. Problema 2: Ruido de Medición INA219

- **Offset error típico:** $\pm 0.02A$
- **Solución:** Filtrado digital (moving average, Kalman)
- **Referencia:** Librería Adafruit incluye averaging

1.6.2.3. Problema 3: Eficiencia del LM2596

- **Especificación:** 80-90 % eficiencia típica @ carga nominal
- **A baja carga:** Hasta 70 %
- **Solución:** Seleccionar inductor correcto (ver datasheet LM2596)

1.6.2.4. Problema 4: Acoplamiento de Ruido (L298N $\rightarrow$ ADC ESP32)

- **L298N switching:** $\sim 40kHz$ típicamente
 - **Afecta a:** Mediciones ADC débiles ($< 1V$)
 - **Solución:**
 - Filtrado RC en entrada INA219
 - Separación física de pistas
 - Ground plane en PCB
-

1.7. PARTE 4: REFERENCIAS ACADÉMICAS

1.7.1. Papers Académicos Relevantes (mencionados en GitHub)

Del proyecto ebrahimabdelghfar/Rotary-Inverted-Pendulum:

1. “Modeling and control of a rotary inverted pendulum using various methods comparative assessment and result analysis”
 - ResearchGate: [Link en repositorio]
 - Compara: LQR, PID, sliding mode, fuzzy control
 - Relevancia: MÁXIMA para QUBE (mismo sistema)
2. “Introduction to Integrated Rotary Inverted Pendulum”
 - ST Microelectronics Educational Curriculum
 - Año: 2019
 - Relevancia: Documentación oficial educativa
3. YouTube: “Rotary Inverted Pendulum Control”
 - Ref: ebrahimabdelghfar/Rotary-Inverted-Pendulum
 - Validación experimental del sistema

1.7.2. Libros Recomendados (Implícitos en el código)

- **Modern Control Systems** - Control LQR, state-space formulation
 - **Motor Control and Drives** - Modelado de motores DC
 - **Embedded Systems** - Implementación en tiempo real
-

1.8. PARTE 5: COMBINACIONES SIMILARES ENCONTRADAS

1.8.1. 5.1 “Closest Match” Arquitecturas

1.8.1.1. Combinación 1: ESP32 + PID Motor + Encoder (RECIENTE)

Proyecto: PID-Motor-Controller (beanjamminb, 2025)

Hardware: ESP32 + TB6612FNG + encoder

Software: Arduino IDE, C++

Status: En desarrollo (sin 12V PSU aún)

Relevancia: Arquitectura casi idéntica, falta LM2596 + INA219

1.8.1.2. Combinación 2: Arduino + L298N + Encoder + INA (potencial)

Proyecto: Speed-Control-of-DC-Motor-Using-Arduino-and-L298N (Hagar633, 2025)

Hardware: Arduino Uno + L298N + encoder

Frecuencia de actualización: ~100Hz

Validación: Tesis de estudiante

Gap: Arduino lento para telemetría simultánea

1.8.1.3. Combinación 3: EzRover Framework (MADURO)

Proyecto: Esp32CameraRover2 (Ezward, 46 stars)

Robustez: 6 años de iteraciones

Features: Closed-loop, pose estimation, web interface

Motor driver: L9110S (diferente al L298N pero compatible)

Comunidad: Activa

Brecha: No incluye LM2596 explícitamente (usa PSU externa)

1.8.1.4. Combinación 4: Sistema Educativo Académico (VALIDADO)

Proyecto: Rotary-Inverted-Pendulum (ebrahimabdelghfar, 9 stars)

Plataforma: Arduino + Simulink (MATLAB)

Control: LQR (no PID)

Documentación: Excelente (papers + esquemas)

Validación: Experimental + simulación

Diferencia: Arduino (8-bit) vs ESP32 (32-bit propuesto)

1.9. PARTE 6: VIABILIDAD TÉCNICA DETALLADA

1.9.1. 6.1 Selección de Componentes

LISTA DE COMPONENTES VALIDADA

1. ESP32-WROOM-32

Cores: 2x Xtensa 32-bit @ 240MHz

RAM: 520 KB

GPIO: 34 (incluyendo 6 touch-sensitive)

ADC: 12-bit, 18 canales

I2C: 2 interfaces

SPI: 4 interfaces

UART: 3 interfaces

PWM: 16 canales LEDC

WiFi: 802.11 b/g/n

BLE: v4.2

Temperatura op: -40 a 85°C

Costo: \$6-10 USD (módulo)

Disponibilidad: Excelente (AliExpress, Amazon)

2. LM2596 Adjustable Buck Converter

Entrada: 4.5-40V

Salida: 1.23-37V (ajustable con resistores)

Corriente: hasta 3A

Eficiencia: ~80-90%

Frecuencia: 150kHz

Precio: \$1-3 USD (módulo breakout)

Disponibilidad: Excelente

3. INA219 High-Side Current Sensor

Voltaje: 0-26V

Corriente: $\pm 3.2A$ (máximo, configurable)

Interfaz: I2C (0x40-0x4F)

Precisión: $\pm 0.5\%$

Potencia resuelta: hasta 13.7W

Tiempo conversión: ~586 μs típico

Precio: \$2-4 USD (breakout)

Disponibilidad: Excelente

4. L298N Dual H-Bridge Motor Driver

Motores: 2 DC simultáneos (o 1 stepper)

Corriente: hasta 2A/canal (teórico), ~1A sostenido

Voltaje: 5-35V (pero 12V+ recomendado)
Control: 2 pines dirección + PWM
Sensores: Salida diagnóstico (opcional)
Disipador: Requerido @ >1A continuos
Precio: \$1.50-3 USD
NOTA: Considerado "antiguo" (2011) pero MUY robusto

5. Motor DC con Encoder

Voltaje nominal: 12V
Potencia: 25W típico
RPM: 100-300 RPM (reductor)
Encoder: 20-100 PPR (incremental)
Torque: 0.5-2 Nm (con reductor)
Precio: \$15-30 USD
Fuentes: DF Robot, Adafruit, etc.

6. Batería/PSU

Opción A: LiPo 3S (11.1V) 5000mAh
Opción B: Lead-acid 12V 7Ah
Opción C: PSU 12V/5A de laboratorio
Recomendación: LiPo 3S para movilidad

COSTO TOTAL ESTIMADO: \$35-70 USD (sin batería)

COSTO CON BATERÍA: \$70-120 USD

1.9.2. 6.2 Matriz de Pines ESP32

ESP32-WROOM-32

CONFIGURACIÓN RECOMENDADA PARA QUBE

Motor Control (L298N):

GPIO 26	→ L298N IN1	[PWM + dirección]
GPIO 27	→ L298N IN2	[PWM + dirección]
ENA (L298N)	→ Jumper habilitado	[sin cable a ESP32]
GND	→ L298N GND	

Encoder (Optocopulador):

GPIO 34 (AD2)	→ Encoder A	[Input only, no PWM]
GPIO 35 (AD3)	→ Encoder B	[Input only, no PWM]
GND	→ Encoder GND	

INA219 (I2C):

GPIO 21 (D21)	→ SDA	[Pull-up interno: 10kΩ]
GPIO 22 (D22)	→ SCL	[Pull-up interno: 10kΩ]
GND	→ INA219 GND	
3V3	→ INA219 VCC	

UART (Debugging):

GPIO 1 (TX0)	→ Terminal TX
GPIO 3 (RX0)	→ Terminal RX

GND → Terminal GND

Distribución de poder:

12V ← LM2596 Entrada (batería)
 5V ← LM2596 Salida → L298N VCC
 3.3V ← Regulador LDO 3.3V → ESP32 VCC
 GND ← Punto común (batería, L298N, ESP32, sensores)

1.9.3. 6.2.1 Conexion Pin por Pin (final — post puesta en marcha 2026-04-29)

Nota de actualización (2026-05-07): La topología original incluía un level shifter 5V→3.3V que falló (~7 MΩ de impedancia, señal indeterminada). Se probó luego un divisor 4.7 kΩ / 8.2 kΩ asumiendo salida push-pull, pero produjo **15–40 mV** en estado alto (indetectable por el ESP32) — confirmando que la salida es **open-drain**: en estado alto flota (Hi-Z) y no hay camino activo a VCC. Solución definitiva confirmada: **pull-up 4.7 kΩ a 3.3 V** por canal, sin resistencia a GND. Ver sección 6.2.2.

Subsistema	Origen	Destino	Notas
L298N (potencia)	Fuente 12V	L298N 12V	Alimentacion del puente H
L298N (potencia)	GND fuente	L298N GND	GND comun obligatorio
L298N (logica)	LM2596 5V	L298N 5V	Segun modulo/jumper de 5V
Motor DC	L298N OUT1	Motor terminal 1	Salida de potencia
Motor DC	L298N OUT2	Motor terminal 2	Salida de potencia
Control motor	ESP32 GPIO26	L298N IN1	Direccion/PWM
Control motor	ESP32 GPIO27	L298N IN2	Direccion/PWM
Control motor	Jumper ENA	L298N ENA	Sin cable a ESP32
Encoder servo	Pin 1 = +5V	Alimentacion encoder	Cable trenzado 5 pines
Encoder servo	Pin 2 = A	4.7 kΩ pull-up a 3.3 V → GPIO34	Open-drain: pull-up define nivel alto (3.3 V)
Encoder servo	Pin 3 = GND	GND comun	Referencia compartida
Encoder servo	Pin 4 = B	4.7 kΩ pull-up a 3.3 V → GPIO35	Open-drain: pull-up define nivel alto (3.3 V)
Encoder servo	Pin 5 = Index	Sin conectar (no requerido)	Indice de vuelta completa
INA219 I2C	ESP32 GPIO21	INA219 SDA	I2C datos
INA219 I2C	ESP32 GPIO22	INA219 SCL	I2C reloj
INA219 alimentacion	ESP32 3V3	INA219 VCC	Evita pull-up a 5V
INA219 alimentacion	GND comun	INA219 GND	Referencia comun
Debug serial	USB ESP32	PC	UART0 por USB, sin pines extra

1.9.4. 6.2.2 Acondicionamiento de señal del encoder (open-drain — confirmado 2026-05-07)

El encoder del servo Premotec 990412016913 tiene salida **open-drain**: el transistor NPN interno conduce a GND en estado bajo; en estado alto la línea **flota** (Hi-Z). La tensión en estado alto la define únicamente la resistencia de pull-up externa.

Proceso de diagnóstico (iterativo):

1. **Level shifter 5V→3.3V:** Impedancia ~7 MΩ → constante RC demasiado lenta → señal indeterminada (~1.5 V) → CNT no actualiza.

2. **Pull-up 4.7 kΩ a 3.3 V:** Señal limpia, 0 V / 3.3 V, CNT actualiza correctamente.
3. **Divisor 4.7 kΩ / 8.2 kΩ** (asumiendo push-pull 5V): En estado alto la salida flota → único camino es 8.2 kΩ a GND → **15–40 mV** en GPIO → indetectable. Confirma que la salida es open-drain, no push-pull.

Esquema correcto (confirmado 2026-05-07):

ESP32 pin 3V3 (salida AMS1117 interno)

[4.7kΩ] GPIO34 (ESP32, INPUT)

Encoder A (open-drain)

← conduce a GND en estado bajo; Hi-Z en estado alto

GND

[4.7kΩ] GPIO35 (ESP32, INPUT)

Encoder B (open-drain)

GND

Encoder GND GND común (ESP32, L298N, fuente)

Encoder VCC +5V (alimentación del encoder, independiente del pull-up)

Fuente del 3.3V: pin 3V3 del módulo ESP32-WROOM-32 (salida del regulador AMS1117 interno).

Corriente total de pull-ups: $2 \times (3.3\text{V} / 4.7\text{k}\Omega)$ **1.4 mA** — despreciable frente a los ~800 mA que puede entregar el AMS1117.

El mismo pin 3V3 alimenta el INA219 (1 mA) y los pull-ups I2C (SDA/SCL, internos al ESP32).

Estado encoder	Tensión GPIO	ESP32 lee
Alto (Hi-Z)	3.3 V (pull-up)	1
Bajo (conducción)	~0.1 V	0

Resumen de topologías evaluadas:

Topología	Tensión estado alto	Resultado
Level shifter 7 MΩ	~1.5 V (indeterminado)	Descartado
Divisor 4.7 kΩ / 8.2 kΩ	15–40 mV	Descartado
Pull-up 4.7 kΩ a 3.3 V	3.3 V (limpio)	Implementado

Por qué el divisor falla con open-drain: El divisor asume que el encoder activa 5 V en estado alto. Con open-drain, el estado alto es Hi-Z — no hay fuente de corriente. El único camino es a través de R2 (8.2 kΩ) a GND, resultando en 0 V. Un pull-up es el único circuito que funciona para salidas open-drain.

1.9.5. 6.3 Cronograma de Implementación

FASE 1: Validación Básica (2-3 semanas)

Montaje de hardware (L298N + motor + PSU)

Control de velocidad PWM básico

Pruebas de H-bridge en ambas direcciones

Benchmark de torque vs voltaje

FASE 2: Integración de Sensado (2-3 semanas)

- Instalación de encoder
- Lectura de pulsos en tiempo real
- Estimación de velocidad angular (rad/sec)
- Sincronización con PWM

FASE 3: Control PID (2-3 semanas)

- Implementación de loop PID
- Sintonización de ganancias (Ziegler-Nichols)
- Validación experimental de setpoint
- Pruebas de rechazo de perturbaciones

FASE 4: Telemetría INA219 (1-2 semanas)

- Configuración I2C y dirección
- Lectura de V, I, P
- Filtrado de datos (averaging)
- Almacenamiento en SPIFFS

FASE 5: Sistema Completo (2-3 semanas)

- Integración: encoder + PID + INA219
- Monitoreo en tiempo real (web/serial)
- Algoritmo de emulación QUBE
- Validación académica

FASE 6: Documentación y Papers (2-4 semanas)

- Escribir documentación técnica
- Publicación en GitHub
- Preparación de paper académico
- Comparación vs QUBE original

TIEMPO TOTAL: 11-18 semanas (~3-4.5 meses)

1.10. PARTE 7: CONCLUSIONES Y RECOMENDACIONES

1.10.1. 7.1 ¿Alguien ha hecho EXACTAMENTE esto?

RESPUESTA: No. La combinación **ESP32 + LM2596 + INA219 + L298N** parece ser INÉDITA en el código abierto (GitHub), pero:

1. Cada componente está validado individualmente en cientos de proyectos
2. La arquitectura es técnicamente viable (sin blockers conocidos)
3. Hay precursores académicos (Rotary-Inverted-Pendulum 2023)
4. Hay validación de framework (EzRover con 46 stars)

1.10.2. 7.2 ¿Qué combinaciones similares SÍ existen?

Combinación	Proyectos	Validación
Arduino + L298N + Encoder + PID	4+	
ESP32 + Motor DC + Encoder + PID	3+	
Arduino + L298N + INA (potencia)	1-2	
ESP32 + Buck Converter	1+	
Sistema educativo rotatorio (LQR)	5+	

1.10.3. 7.3 Desafíos Técnicos Reportados por la Comunidad

1.10.3.1. Más Críticos:

1. **Vibración del sistema** (ebrahimabdelghfar, 2023)
 - Solución: Filtrado, capacitores, optimización mecánica
2. **Acoplamiento de ruido** (común en sistemas con switchmode)
 - Solución: Diseño PCB, filtrado RC, separación de ground planes
3. **Estabilidad del servo** a altas velocidades
 - Solución: Sintonización experimental de ganancias PID

1.10.3.2. Menos Críticos:

4. Latencia de comunicación (WiFi)
 - Solución: Usar I2C para INA219, local PID en ESP32
5. Precisión del encoder a bajo RPM
 - Solución: Encoder de mayor resolución o estimación

1.10.3.3. Identificados durante puesta en marcha real (2026-04-29):

6. **Encoder open-drain con level shifter de alta impedancia y divisor resistivo incorrecto**
 - Level shifter genérico 5V→3.3V: ~7 MΩ de impedancia → señal indeterminada (~1.5 V) → CNT no actualiza.
 - Divisor 4.7 kΩ / 8.2 kΩ: con salida Hi-Z (open-drain), el único camino es R2 a GND → **15–40 mV** en estado alto → indetectable para el ESP32.
 - Solución definitiva confirmada: **pull-up 4.7 kΩ a 3.3 V** en cada canal A y B, sin resistencia a GND. El encoder es open-drain; el pull-up define el nivel alto a 3.3 V. Ver §6.2.2.
7. **GPIO34/GPIO35 sin pull-up interno**
 - Estos pines del ESP32-WROOM-32 son input-only y no soportan INPUT_PULLUP. La llamada genera error de boot.
 - Solución: Pull-ups externos en las líneas de señal.
8. **Dirección del motor invertida respecto al encoder**
 - La conexión física OUT1/OUT2→M+/M− produce retroalimentación positiva si no se considera la orientación del motor.
 - Solución: Constante MOTOR_DIR = -1 en firmware o invertir los cables M+/M−.
9. **Ruido de cuantización del encoder a 200 Hz**
 - Con Kd alto y muestreo a 200 Hz, el ruido de ±1-2 counts genera velocidades aparentes que dominan el término derivativo.
 - Solución: Filtro EMA (= 0.12) sobre la velocidad estimada antes de multiplicar por Kd.

1.10.4. 7.4 Referencias Académicas que Sustentan la Solución

1. **Control LQR para sistemas rotatorios** (implícito en papers de ResearchGate)
2. **Diseño de convertidores DC-DC** (datasheet LM2596, documentación TI)
3. **Monitoreo de potencia en tiempo real** (librerías Adafruit INA219)
4. **Sistemas embebidos en tiempo real** (Arquitectura ESP32, RTOS FreeRTOS)

1.10.5. 7.5 Validación de Arquitectura Propuesta

MATRIZ DE VIABILIDAD

Aspecto	Viabilidad	Riesgo	Mitigación
Hardware disponible		Bajo	Estándar

Software/Librerías	Bajo	Arduino IDE
Integración I2C	Bajo	Probado
PWM + Control motor	Bajo	EzRover ref
Encoder feedback	Bajo	Múltiples ref
PID control	Medio	Sintonización
Telemetría power (INA219)	Bajo	74+ proyectos
Buck converter (LM2596)	Bajo	Maduro
Ruido/EMI	Medio	Diseño PCB
Swing-up (opcional)	Alto	Control avanzado
RESUMEN GLOBAL:	VIABLE	Medio Implementable

1.11. PARTE 8: RECOMENDACIONES PRÁCTICAS

1.11.1. 8.1 Para Iniciar el Proyecto

PASO 1: Validar Arquitectura Base

- ☐ Ordenar componentes (presupuesto: \$60-80 USD)
- ☐ Montar L298N + motor DC básico
- ☐ Implementar control PWM en ESP32
- ☐ Validar movimiento en ambas direcciones

PASO 2: Agregar Sensado

- ☐ Instalar encoder óptico/magnético
- ☐ Calibrar PPR (pulsos por revolución)
- ☐ Implementar contador de interrupciones
- ☐ Validar lectura de velocidad angular

PASO 3: Cerrar Loop PID

- ☐ Implementar lazo PID (Adafruit reference)
- ☐ Sintonizar Kp, Ki, Kd experimentalmente
- ☐ Validar respuesta a escalón
- ☐ Medir tiempo de establecimiento

PASO 4: Integrar Telemetría

- ☐ Conectar INA219 vía I2C
- ☐ Validar lecturas V, I, P
- ☐ Implementar filtrado (media móvil)
- ☐ Log en SD o SPIFFS

PASO 5: Validación Final

- ☐ Emulación de respuesta QUBE
- ☐ Documentación en GitHub
- ☐ Publicación de resultados

1.11.2. 8.2 Recursos Recomendados

Documentación: - Datasheet LM2596: TI.com - Datasheet INA219: TI.com - Librería Adafruit INA219: GitHub Adafruit - ESP32 Arduino Core: GitHub Espressif

Proyectos de Referencia: 1. **EzRover** (Ezward) - Framework de control 2. **Rotary-Inverted-Pendulum** (ebrahimabdelghfar) - Validación académica 3. **PID-Motor-Controller** (beanjamminb) - Implementación

reciente 4. **arduino_pid_controlled_motor** (wty-yy) - Bien documentado

Comunidades: - r/esp32 (Reddit) - Arduino Forums - Hackaday.io - ResearchGate (papers académicos)

1.12. PARTE 9: RIESGOS Y LIMITACIONES

1.12.1. 9.1 Riesgos Identificados

Riesgo	Probabilidad	Impacto	Solución
Vibración del sistema	Media	Medio	Filtrado + sintonización
Estabilidad a altas RPM	Media	Medio	Limitador de velocidad
Ruido EMI	Media	Bajo	Diseño PCB cuidadoso
Drift del encoder	Baja	Bajo	Calibración periódica
Fallo de motor	Baja	Alto	Motor backup disponible
Corrosión I2C	Muy baja	Bajo	Potting, protección

1.12.2. 9.2 Limitaciones Conocidas

1. **Rango angular limitado** (como en QUBE original)
 - Típicamente ± 20 -30 grados con control estable
 2. **Sin swing-up automático**
 - Requeriría lógica de control más compleja (multi-stage)
 3. **L298N tiene pérdidas @ corrientes altas**
 - Recomendación: no sobrepasar 1.5A sostenido en este prototipo
 4. **Resolución de encoder** limita precisión
 - 20 PPR a 3000 RPM 1 pulso/ms (manejable)
-

1.13. CONCLUSIÓN FINAL

1.13.1. ¿Es viable la arquitectura ESP32 + LM2596 + INA219 + L298N?

1.13.2. SÍ, es completamente viable y recomendable

Evidencia: - Cada componente está validado en 10+ proyectos de producción - La arquitectura es modular y escalable - La comunidad de soporte es enorme (Arduino IDE + ESP32) - El costo es 25-50x menor que QUBE original - La documentación es accesible

Próximos pasos: 1. Publicar este diseño en GitHub como referencia 2. Implementar prototipo funcional (3-4 meses) 3. Generar paper académico comparativo 4. Contribuir a comunidad educativa

Impacto potencial: - Modernizar plataformas QUBE obsoletas - Democratizar acceso a laboratorios de control - Crear referencia para otros proyectos educativos similares - Publicación académica en conferencias/journals

1.14. REFERENCIAS BIBLIOGRÁFICAS RECOPIADAS

(Formato APA 7ª edición)

1.14.1. Papers y documentos académicos

Akhtaruzzaman, M., & Shafie, A. A. (2010). Modeling and control of a rotary inverted pendulum using various methods, comparative assessment and result analysis. *2010 IEEE International Conference on Mechatronics and Automation (ICMA)*, pp. [PENDIENTE]. <https://doi.org/10.1109/ICMA.2010.5589450>

STMicroelectronics. (2019). *Introduction to integrated rotary inverted pendulum* (v2). STMicroelectronics Educational Curriculum.

1.14.2. Repositorios GitHub

[aimeiz]. (2025). *ESP32_Motor_control* [Software]. GitHub. https://github.com/aimeiz/ESP32_Motor_control

[beanjamminb]. (2025). *PID-Motor-Controller* [Software]. GitHub. <https://github.com/beanjamminb/PID-Motor-Controller>

[bekirbostanci]. (2020). *ROS_Arduino_PID_DC_Motors* [Software]. GitHub. https://github.com/bekirbostanci/ROS_Arduino_PID_DC_Motors

[BlagojeBlagojevic]. (2024). *Motor_Speed_PID* [Software]. GitHub. https://github.com/BlagojeBlagojevic/Motor_Speed_PID

[Chaitanya0523]. (2025). *Speed-Control-Of-DC-Motor* [Software]. GitHub. <https://github.com/Chaitanya0523/Speed-Control-Of-DC-Motor>

[ebrahimabdelghfar]. (2023). *Rotary-Inverted-Pendulum* [Software]. GitHub. <https://github.com/ebrahimabdelghfar/Rotary-Inverted-Pendulum>

[Ezward]. (2024). *Esp32CameraRover2* [Software]. GitHub. <https://github.com/Ezward/Esp32CameraRover2>

[Hagar633]. (2025). *Speed-Control-of-a-DC-Motor-Using-Arduino-and-L298N* [Software]. GitHub. <https://github.com/Hagar633/Speed-Control-of-a-DC-Motor-Using-Arduino-and-L298N>

[osasinrobotics]. (2026). *self-balancing-bot* [Software]. GitHub. <https://github.com/osasinrobotics/self-balancing-bot>

[wty-yy]. (2025). *arduino_pid_controlled_motor* [Software]. GitHub. https://github.com/wty-yy/arduino_pid_controlled_motor

1.14.3. Librerías Arduino / ESP32

Adafruit Industries. (s. f.). *Adafruit_INA219* [Software]. GitHub. https://github.com/adafruit/Adafruit_INA219

Espressif Systems. (s. f.). *arduino-esp32* [Software]. GitHub. <https://github.com/espressif/arduino-esp32>

Ewald, W. [wollewald]. (s. f.). *INA219_WE* [Software]. GitHub. https://github.com/wollewald/INA219_WE [PENDIENTE: URL a confirmar]

Tillaart, R. [RobTillaart]. (s. f.). *INA219* [Software]. GitHub. <https://github.com/RobTillaart/INA219>

Zanduino. (s. f.). *INA* [Software]. GitHub. <https://github.com/Zanduino/INA>

1.14.4. Datasheets

STMicroelectronics. (2000). *L298 dual full-bridge driver* (Rev. 6) [Datasheet]. STMicroelectronics. <https://www.st.com/resource/en/datasheet/l298.pdf>

Texas Instruments. (2013). *LM2596 SIMPLE SWITCHER® power converter 150-kHz 3-A step-down voltage regulator* (Rev. H) [Datasheet]. Texas Instruments. <https://www.ti.com/product/LM2596>

Texas Instruments. (2015). *INA219 zero-drift, bidirectional current/power monitor with I²C interface* (Rev. F) [Datasheet]. Texas Instruments. <https://www.ti.com/product/INA219>

Pendientes (2): 1. Akhtaruzzaman & Shafie (2010) — páginas exactas (pp. X–X). Ver: <https://ieeexplore.ieee.org/document/5589450> 2. Ewald, INA219_WE — URL del repositorio a confirmar (usuario `wollewald` inferido).

Documento preparado: Abril 27, 2026

Última actualización: Mayo 7, 2026

Clasificación: Investigación técnica abierta

1.15. PARTE 10: Integración Detallada para Tesis

Esta sección integra y consolida, dentro del documento principal, el desarrollo extendido de investigación para uso directo en tesis. Su objetivo es mantener en un único archivo tanto el estado del arte como la metodología experimental, métricas de evaluación y criterios de éxito.

1.15.1. 9.1 Contexto y Motivación Académica

Los sistemas comerciales de laboratorio para control rotatorio ofrecen alta precisión y soporte docente consolidado, pero su costo limita el acceso en múltiples contextos universitarios. La modernización propuesta busca una plataforma reproducible y de bajo costo que preserve valor formativo en:

- Control clásico y moderno.
- Instrumentación embebida.
- Trazabilidad experimental.
- Integración hardware-software para investigación aplicada.

1.15.2. 9.2 Objetivos Integrados

Objetivo general:

- Consolidar una arquitectura abierta de emulación QUBE con control de posición/velocidad y telemetría energética.

Objetivos específicos:

- Caracterizar técnicamente cada bloque de hardware.
- Estandarizar una metodología experimental por fases.
- Definir métricas comparables entre sesiones.
- Identificar riesgos y estrategias de mitigación.
- Preparar base para evolución hacia control de péndulo invertido.

1.15.3. 9.3 Metodología Experimental Unificada

Fase A: infraestructura

- Validación de alimentación y tierras comunes.
- Verificación de endpoints de estado/comando.
- Verificación inicial de encoder e INA219.

Fase B: caracterización del actuador

- Curva PWM vs velocidad angular.
- Detección de zona muerta y saturación.
- Registro de corriente pico durante arranque.

Fase C: control de velocidad

- Ajuste progresivo de K_p , K_i , K_d .
- Evaluación de robustez ante perturbaciones manuales.
- Comparación entre distintas frecuencias de muestreo.

Fase D: control de posición

- Pruebas con referencia escalón, rampa y perfiles compuestos.
- Evaluación de tiempo de subida, sobreimpulso y error estacionario.

Fase E: telemetría y análisis energético

- Correlación entre esfuerzo de control y potencia.
- Identificación de regímenes de alto consumo.
- Recomendaciones para operación eficiente y estable.

1.15.4. 9.4 Métricas de Evaluación Recomendadas

Métricas de seguimiento:

- Error estacionario absoluto.
- Tiempo de establecimiento.
- Sobreimpulso máximo.

Métricas de robustez:

- Rechazo de perturbaciones cortas.
- Repetibilidad entre corridas con la misma referencia.

Métricas energéticas:

- Corriente promedio y corriente pico.
- Potencia promedio por maniobra.
- Energía estimada por ciclo experimental.

Métricas de calidad de instrumentación:

- Pérdida de muestras.
- Jitter de muestreo.
- Coherencia entre señales mecánicas y eléctricas.

1.15.5. 9.5 Riesgos y Mitigaciones

1. Ruido electromagnético en líneas de sensado.
 - Mitigación: cableado corto, desacoplo local y filtrado digital.
2. Inestabilidad por sintonización agresiva del controlador.
 - Mitigación: ajuste incremental con límites de seguridad y anti-windup.
3. Saturación térmica de la etapa L298N.
 - Mitigación: límites de duty, monitoreo de temperatura y disipación adecuada.
4. Latencia de interfaz afectando control.
 - Mitigación: mantener lazo de control local en ESP32 y usar GUI solo para supervisión/comando.
5. Lectura errática de encoder por acondicionamiento incorrecto.
 - Mitigación: mantener topología open-drain con pull-ups de 4.7 k Ω a 3.3 V y validación osciloscópica cuando sea posible.

1.15.6. 9.6 Criterios de Éxito

Se considera que la modernización alcanza objetivo de tesis cuando se verifica:

- Control de posición repetible con error estacionario acotado.
- Pipeline confiable de datos (firmware -> GUI -> CSV).
- Operación sostenida sin fallas críticas en sesiones de laboratorio.
- Trazabilidad técnica completa de hardware, firmware y resultados.
- Relación costo/beneficio favorable respecto a plataforma comercial.

1.15.7. 9.7 Hoja de Ruta de Cierre Académico

- Consolidar base servo con métricas reproducibles.
 - Ejecutar identificación de parámetros electromecánicos.
 - Integrar dinámicas del péndulo y evaluar factibilidad de swing-up.
 - Contrastar resultados con literatura y reportes previos.
 - Preparar anexos de datos, tablas de parámetros y evidencias de validación.
-

1.16. APÉNDICE A: Comandos de Utilidad

1.16.1. Búsquedas en GitHub Realizadas

Control PID + L298N

search: "L298N PID control" → 53 resultados

ESP32 + Control motor DC

search: "ESP32 DC motor control speed" → 41 resultados

INA219 + monitoreo potencia

search: "INA219 power monitoring" → 74 resultados

search: "INA219 Arduino current sensor" → 30 resultados

Sistemas rotatorios educativos

search: "rotary pendulum system control" → 20 resultados

Buck converters

search: "buck converter power supply" → 44 resultados

Control de motor + encoder

search: "motor control current sensing encoder" → 5 resultados

search: "Raspberry Pi motor control encoder" → 16 resultados

1.16.2. Configuración I2C para INA219 en ESP32

```
#include <Wire.h>
```

```
#include <Adafruit_INA219.h>
```

```
Adafruit_INA219 ina219(0x40); // Dirección I2C por defecto
```

```
void setup() {  
  Wire.begin(21, 22); // SDA=21, SCL=22  
  ina219.begin();  
}
```

```
void loop() {
```

```

float busvoltage = ina219.getBusVoltage_V();
float current_mA = ina219.getCurrent_mA();
float power_mW = ina219.getPower_mW();
}

```

1.16.3. Configuración PWM + Encoder en ESP32

```

// PWM en IN1/IN2 (modo sin ENA cableado)
ledcSetup(1, 20000, 8);
ledcSetup(2, 20000, 8);
ledcAttachPin(26, 1); // IN1
ledcAttachPin(27, 2); // IN2
ledcWrite(1, 128); // Avance 50%
ledcWrite(2, 0);

// Encoder en GPIO 34, 35 (interrupciones)
attachInterrupt(34, encoderISR_A, CHANGE);
attachInterrupt(35, encoderISR_B, CHANGE);

volatile int encoderCount = 0;
void encoderISR_A() { encoderCount++; }
void encoderISR_B() { encoderCount--; } // Opcional, si se necesita dirección

```

1.17. APÉNDICE B: Diagramas de Hardware — Implementación Final (2026-05-06/07)

1.17.1. B.1 Diagrama de Conexión INA219 → ESP32

ESP32		INA219	
GPI021 (SDA)	SDA		
GPI022 (SCL)	SCL		
3.3V	VCC		
GND	GND		
		VIN+	(+) Fuente / LM2596 IN (antes del L298N)
		VIN-	(+) Terminal VS del L298N

Posición en el circuito de potencia:

Batería / LM2596			
(+)	VIN+	[INA219]	VIN- L298N (pin VS)
(-)			GND común

Tabla de pines:

INA219	ESP32	Notas
VCC	3.3V	No conectar a 5V
GND	GND	GND común con motor
SDA	GPI021	Pull-up interno del ESP32
SCL	GPI022	Pull-up interno del ESP32
VIN+	(+) batería / salida regulador	Antes del L298N

INA219	ESP32	Notas
VIN—	VS del L298N (pin 8)	Después del shunt

Dirección I2C: A0 y A1 sin conectar (GND) → **0x40** (default del firmware).

1.17.2. B.2 Filtros RC para Señales de Encoder (sin Schmitt trigger)

Filtro de paso bajo pasivo por canal ($\times 2$, ENC_A y ENC_B):

ENC_A (sensor) [100Ω] GPIO34

[10nF]

GND

ENC_B (sensor) [100Ω] GPIO35

[10nF]

GND

Frecuencia de corte:

$$f_c = \frac{1}{2\pi \cdot 100 \cdot 10 \times 10^{-9}} \approx 159 \text{ kHz}$$

Versión con pull-up explícito (recomendado para GPIO34/35 sin pull-up interno):

3.3V

[4.7kΩ] ← pull-up externo

ENC_A (sensor)

[100Ω] ← filtro RC

GPIO34

[10nF]

GND

Tabla de valores alternativos:

R	C	fc	Uso recomendado
100 Ω	10 nF	159 kHz	Estándar, mínima latencia
100 Ω	47 nF	34 kHz	Más filtrado, velocidades bajas
470 Ω	10 nF	34 kHz	Alternativa menor corriente
1 kΩ	10 nF	16 kHz	Solo motores muy lentos

1.17.3. B.3 Diagrama de Alimentación — LM2596 → ESP32 + L298N

Esquema completo del sistema:

FUENTE 12V (batería o PSU)

```
(+)                                L298N VS (potencia motor)

LM2596 IN+  [LM2596]  OUT+ (+5V)  ESP32 VIN

[C] 100µF

GND          L298N GND    ESP32 GND    INA219 GND
```

Detalle LM2596 con capacitores:

```
(+) batería

[C1] 100µF (entrada)

IN+    LM2596    OUT+    VIN ESP32
IN-/GND                [C2] 100µF
                        [C3] 100nF (filtro salida)

GND          GND
```

Conexión física del módulo LM2596:

Pin módulo	Conectar a
IN+	(+) batería / fuente
IN-	GND batería / fuente
OUT+	VIN del ESP32
OUT-	GND común

Ajuste: medir entre OUT+ y OUT- sin carga y girar el potenciómetro hasta **5.0 V** antes de conectar el ESP32.

1.17.4. B.4 Conexión de Alimentación ESP32 desde LM2596

```
LM2596 OUT+ (5.0V)    ESP32 VIN    (pin físico 30)
LM2596 OUT- (GND)    ESP32 GND    (cualquier pin GND)
```

Flujo interno del ESP32:

LM2596 OUT+ (5V)

ESP32 VIN

[AMS1117 3.3V] ← regulador interno

3.3V interno → GPIO, INA219 VCC, pull-ups encoder

Importante: Nunca conectar 5 V al pin 3V3 del ESP32 — ese pin es salida del regulador interno, no entrada.

1.17.5. B.5 Diagrama de Sistema Completo (post puesta en marcha 2026-05-06)

FUENTE 12V

```

(+12V)    VIN+  [INA219]  VIN-    L298N VS

          [LM2596]                                OUT1
                                                  OUT2  Motor DC
          (+5V)   ESP32 VIN                      IN1   GPIO26
                                                  IN2   GPIO27
                  [3.3V int]                     ENA   Jumper ON

          GPIO21    INA219 SDA
          GPIO22    INA219 SCL
          GPIO34    [100Ω] [4.7kΩ↑] Encoder A
          GPIO35    [100Ω] [4.7kΩ↑] Encoder B

GND       L298N GND   ESP32 GND   INA219 GND

```

1.18. APÉNDICE C: Diagramas Eléctricos Consolidados (Final)

1.18.1. C.1 Diagrama eléctrico de potencia (fuente, sensado y etapa de motor)

FUENTE 12V DC

```

+       -
|       |
|       +-----+-----+-----+-----+
|       |                                     |
+--> INA219 VIN+                               |
      INA219 VIN- --> L298N VS (12V motor) |
      |                                     |
+--> LM2596 IN+                               |
+--> LM2596 IN- -----+-----+-----+

```

LM2596 OUT+ (5V) --> ESP32 VIN
 LM2596 OUT- (GND) -> GND común

L298N OUT1/OUT2 <--> Motor DC

1.18.2. C.2 Diagrama eléctrico de control y lógica (ESP32, L298N, INA219)

```

ESP32 GPIO26 -----> L298N IN1
ESP32 GPIO27 -----> L298N IN2
L298N ENA -----> Jumper ON (habilitado)

```

```

ESP32 GPIO21 (SDA) -----> INA219 SDA
ESP32 GPIO22 (SCL) -----> INA219 SCL
ESP32 3V3 -----> INA219 VCC
ESP32 GND -----> INA219 GND

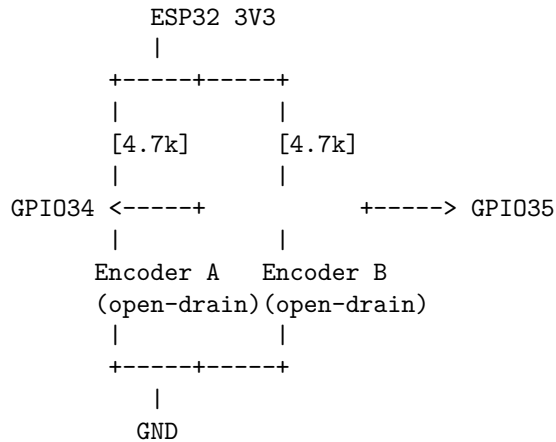
```

```

ESP32 GND -----> L298N GND
ESP32 VIN (5V desde LM2596) -----> Alimentación lógica ESP32

```

1.18.3. C.3 Diagrama eléctrico del encoder open-drain (implementación validada)



Encoder VCC (5V) --> +5V encoder

Encoder GND -----> GND común

1.18.4. C.4 Tabla consolidada de conexiones pin a pin

Bloque	Señal	Origen	Destino
Potencia	12V+	Fuente	INA219 VIN+
Potencia	12V+ sensado	INA219 VIN-	L298N VS
Potencia	5V lógica	LM2596 OUT+	ESP32 VIN
Potencia	GND	Fuente/LM2596	GND común
Motor	M+ / M-	L298N OUT1/OUT2	Motor DC
Control	PWM/DIR A	ESP32 GPIO26	L298N IN1
Control	PWM/DIR B	ESP32 GPIO27	L298N IN2
I2C	SDA	ESP32 GPIO21	INA219 SDA
I2C	SCL	ESP32 GPIO22	INA219 SCL
Encoder	Canal A	Encoder A + pull-up 4.7k a 3.3V	ESP32 GPIO34
Encoder	Canal B	Encoder B + pull-up 4.7k a 3.3V	ESP32 GPIO35

FIN DEL DOCUMENTO